

Guía para desarrolladores - Sábana digital de enfermería HUDN

Esneyder Jesús Ibarra Rosero

Universidad Mariana

Facultad de Ingeniería

Programa de Ingeniería de Sistemas

San Juan de Pasto

2026

Guía para desarrolladores - Sábana digital de enfermería HUDN

Esneyder Jesús Ibarra Rosero

Noveno Semestre

Docente: Wilson Castillo

Universidad Mariana

Facultad de Ingeniería

Programa de Ingeniería de Sistemas

Práctica Empresarial

2026

Tabla de Contenido	4
1. Introducción	6
2. Tecnologías Utilizadas	7
3. Estructura del Proyecto	8
4. Arquitectura (Clean Architecture + Riverpod)	10
4.1 Capa de Dominio (Domain)	10
4.2 Capa de Datos (Data)	10
4.3 Capa de Presentación (Presentation)	10
4.4 Capa de Aplicación (Application)	10
Flujo de datos	10
5. Conexión con Firebase	11
5.1 Inicialización	11
5.2 firebase_options.dart	11
5.3 Estructura de Firestore	11
6. Modelos de Datos (Freezed)	12
6.1 Ejemplo: Ingreso	12
6.2 Ejemplo: ProcedimientoEspecial	12
7. Repositorios y Firestore	13
7.1 Ejemplo: FirebaseNutricionRepository	13
8. Providers de Riverpod	14
8.1 Ejemplo: StreamProvider en tiempo real	14
8.2 Ejemplo: Provider de acción (CRUD)	14
9. Módulos Implementados	15
10. Generación de Reportes PDF	16
10.1 Flujo de Generación	16
10.2 Ejemplo de uso	16
10.3 Fuentes	16
11. Configuración del Entorno de Desarrollo	17
11.1 Requisitos	17
11.2 Pasos	17
Clonar el repositorio	17
Instalar dependencias	17
Configurar Firebase	17
Ejecutar en Android	17
Ejecutar en Web	17
Build APK	17
Build Web + Deploy	17
12. Buenas Prácticas	18

12.1 Commits	18
12.2 Ramas (Git Flow)	18
12.3 Documentación	18
12.4 Pruebas	18
13. Posibles Mejoras Futuras	19
14. Enlaces Útiles	20

1. Introducción

Sábana Digital de Enfermería es una aplicación móvil/web desarrollada en Flutter que reemplaza el formato físico de registro de enfermería en UCI por una solución digital. La aplicación permite registrar, consultar y exportar todos los datos clínicos de pacientes en cuidados intensivos, generando un reporte PDF con el formato tradicional de sábana de enfermería.

Aplicación web desplegada en: <https://sabana-digital-prueba.web.app>

[ESPACIO: Insertar aquí el link de descarga del APK (Drive o Git)]

2. Tecnologías Utilizadas

Componente	Tecnología
Lenguaje	Dart 3.x
Framework	Flutter 3.35.7 (canal stable)
Estado	Riverpod (hooks_riverpod + flutter_riverpod)
Backend	Firebase (Firestore, Auth, Hosting)
PDF	pdf (dart/pdf) + printing (flutter_printing)
Serialización	Freezed + json_serializable
Firmas digitales	signature (flutter)
Base de datos	Cloud Firestore (NoSQL)
Autenticación	Firebase Auth (email/password)
Hosting Web	Firebase Hosting

3. Estructura del Proyecto

El proyecto sigue una estructura feature-first dentro de Clean Architecture. Cada funcionalidad (feature) es un módulo independiente con sus propias capas.

registro_uci/

```
|— lib/
|   |— main.dart          # Punto de entrada
|   |— firebase_options.dart # Configuración de Firebase
|   |— app.dart           # Widget raíz con router
|   |— common/            # Componentes compartidos
|   |   |— components/    # Widgets reutilizables
|   |   |— providers/     # Providers globales
|   |   |— themes/        # Temas (colores, fuentes)
|   |— constants/         # Constantes globales
|   |— features/          # Módulos (Clean Architecture)
|   |   |— auth/          # Autenticación
|   |   |— ingresos/      # Gestión de ingresos
|   |   |— registros_diarios/ # Observaciones por turno
|   |   |— monitorias_hemodinamicas/ # Signos vitales
|   |   |— balance_liquidos/ # Balance de líquidos
|   |   |— nutricion/     # Nutrición
|   |   |— control_riesgos/ # Control de riesgos
|   |   |— antibioticos/   # Antibióticos
|   |   |— cateteres/     # Catéteres
|   |   |— sondas/        # Sondas y drenes
|   |   |— marcapasos/     # Marcapasos
|   |   |— procedimientos_especiales/ # Procedimientos
|   |   |— control_sedacion/ # Escala RASS
|   |   |— control_cambio_posicion/ # Cambio de posición
|   |   |— monitorias_hemodinamicas/glasgow/ # Glasgow
```

- | | | └─ observaciones_extras/ # Lab, cultivos, firmas
- | | └─ reportes/ # Generación de PDF
- | └─ pages/ # Pantallas (ruteo)
- | | └─ ingreso/ # CRUD de ingresos
- | | └─ nutricion/ # Pantalla nutrición
- | | └─ control_riegos/ # Pantalla control riesgos
- | └─ procedimientos_especiales/ # Pantalla procedimientos
- └─ assets/
 - | └─ images/ # Logo, imágenes
- └─ fonts/
 - | └─ Plus_Jakarta/ # Fuentes tipográficas
- └─ android/ # Configuración nativa Android
- └─ test/ # Pruebas unitarias
- └─ pubspec.yaml # Dependencias
- └─ firebase.json # Config Firebase + Hosting

4. Arquitectura (Clean Architecture + Riverpod)

La aplicación implementa Clean Architecture con 4 capas, separando claramente las responsabilidades de cada componente. Además, se usa Riverpod para la gestión de estado.

4.1 Capa de Dominio (Domain)

Contiene las entidades de negocio y reglas de la aplicación.

- `models/` — Clases de datos con Freezed (ej: Ingreso, RegistroNutricional)
- No depende de ninguna otra capa ni de frameworks externos.

4.2 Capa de Datos (Data)

Implementa la comunicación con fuentes externas (Firestore).

- `dto/` — Objetos de transferencia de datos (`CreateXDto`, `UpdateXDto`)
- `repositories/` — Implementaciones concretas de Firebase
- `providers/` — Providers de Riverpod para exponer datos a la UI
- `abstract_repositories/` — Interfaces/contratos abstractos

4.3 Capa de Presentación (Presentation)

Contiene los widgets, controladores y lógica de UI.

- `pages/` — Pantallas completas
- `widgets/` — Componentes de UI reutilizables
- `controllers/` — Lógica de negocio asociada a la UI (`AsyncNotifier`)

4.4 Capa de Aplicación (Application)

Servicios de aplicación que orquestan operaciones complejas.

- Ejemplo: PdfReporteService, ReporteDataCollector

Flujo de datos

UI → Provider (Riverpod) → Repository → Firestore (Cloud)

La UI escucha cambios a través de StreamProvider o FutureProvider.

Cuando un dato cambia en Firestore, el StreamProvider emite un nuevo valor y la UI se reconstruye automáticamente.

5. Conexión con Firebase

Firebase se inicializa en main.dart mediante `Firebase.initializeApp()` usando `firebase_options.dart` generado por FlutterFire CLI.

5.1 Inicialización

```
void main() async {  
  
  WidgetsFlutterBinding.ensureInitialized();  
  
  await Firebase.initializeApp(  
  
    options: DefaultFirebaseOptions.currentPlatform,  
  
  );  
  
  runApp(const ProviderScope(child: SabanaApp()));  
  
}
```

5.2 firebase_options.dart

Archivo generado automáticamente por flutterfire configure. Contiene las credenciales de Firebase para cada plataforma (Android, Web).

5.3 Estructura de Firestore

La base de datos está organizada en colecciones y subcolecciones:

ingresos/{idIngreso}

├── registrosDiarios/{id}/ # Observaciones por turno

│ ├── monitoriasHemodinamicas/

│ ├── controlDeRiesgos/

│ ├── controlesSedacion/

│ ├── cambiosPosicion/

│ ├── glasgow/

│ ├── balancesDeLiquidos/{id}/

│ │ ├── administrados/

│ │ └── eliminados/

│ ├── listaTratamientos/

│ └── necesidades/

├── cateteres/

├── marcapasos/

├── sondas/

├── nutricion/

├── procedimientosEspeciales/

├── tratamientosAntibioticos/

├── pagina6/ # → observaciones_extras/

└── firmas/

6. Modelos de Datos (Freezed)

Los modelos se definen con Freezed para obtener inmutabilidad, copyWith, serialización JSON y comparación por valor de forma automática.

6.1 Ejemplo: Ingreso

```
@freezed
class Ingreso with _$Ingreso {
  const factory Ingreso({
    required String idIngreso,
    required String nombrePaciente,
    required String diagnosticoIngreso,
    required String diagnosticoActual,
    required DateTime fechaIngreso,
    required double peso,
    required int talla,
    required String cama,
    required Sala sala,
    // ... más campos
  }) = _Ingreso;
  factory Ingreso.fromJson(Map<String, dynamic> json,
    {required String id}) => _IngresoFromJson(json, id);
}
```

6.2 Ejemplo: ProcedimientoEspecial

```
@freezed
class ProcedimientoEspecial with _$ProcedimientoEspecial {
  const factory ProcedimientoEspecial({
    required String idProcedimiento,
    required String nombreProcedimiento,
    required String estado, // Por realizar, Realizado, Reportado
  }) = _ProcedimientoEspecial;
}
```

```
String? medicamentoInfusion,  
String? dosisInfusion,  
}) = _ProcedimientoEspecial;  
  
factory ProcedimientoEspecial.fromJson(  
    Map<String, dynamic> json, {required String id}) =>  
    _$ProcedimientoEspecialFromJson(json, id);  
}
```

7. Repositorios y Firestore

Cada módulo tiene un repositorio abstracto (interface) y una implementación concreta para Firebase. Los repositorios encapsulan toda la lógica de acceso a datos.

7.1 Ejemplo: FirebaseNutricionRepository

```
class FirebaseNutricionRepository implements NutricionRepository {  
    final _firestore = FirebaseFirestore.instance;  
  
    Future<List<RegistroNutricional>> getRegistrosNutricionales(  
        String idIngreso,  
    ) async {  
        final snap = await _firestore  
            .collection('ingresos').doc(idIngreso)  
            .collection('nutricion')  
            .orderBy('hora', descending: true).get();  
        return snap.docs.map((doc) =>  
            RegistroNutricional.fromJson(doc.data(), id: doc.id)  
        ).toList();  
    }  
  
    Future<void> createRegistroNutricional(  
        String idIngreso, CreateRegistroNutricionalDto dto,  
    ) async {  
        await _firestore  
            .collection('ingresos').doc(idIngreso)  
            .collection('nutricion').doc()  
            .set(dto.toMap());  
    }  
}
```

8. Providers de Riverpod

Riverpod es el manejador de estado. Se usan los siguientes tipos de providers:

Tipo	Uso
Provider<T>	Provee una instancia singleton (ej: repositorio, servicio)
FutureProvider.family<T, P>	Operación asíncrona parametrizada (ej: obtener ingreso por ID)
StreamProvider.family<T, P>	Stream en tiempo real parametrizado (ej: lista de procedimientos)
AsyncNotifierProvider	Manejo de estado mutable con operaciones asíncronas (CRUD)
StateNotifierProvider	Similar a AsyncNotifier, para estados complejos

8.1 Ejemplo: StreamProvider en tiempo real

```
final procedimientoStreamProvider =  
  StreamProvider.family<List<ProcedimientoEspecial>, String>(  
    (ref, idIngreso) {  
      final repo = ref.watch(procedimientoRepositoryProvider);  
      return repo.getProcedimientosDeRegistro(idIngreso);  
    },  
  );  
  
// Uso en UI:  
final procedimientosAsync = ref.watch(procedimientoStreamProvider(idIngreso));
```


8.2 Ejemplo: Provider de acción (CRUD)

```
final procedimientoActionProvider = Provider<ProcedimientoActions>((ref) {  
    return ProcedimientoActions(ref.watch(procedimientoRepositoryProvider));  
});  
  
// Uso:  
ref.read(procedimientoActionProvider).addProcedimiento(  
    idIngreso, nombre, estado: 'Por realizar'  
);
```

9. Módulos Implementados

Módulo	Ubicación	Descripción
Autenticación	lib/features/auth/	Inicio de sesión con Firebase Auth, roles de usuario
Ingresos	lib/features/ingresos/	CRUD de pacientes, datos personales, diagnóstico
Registros Diarios	lib/features/registros_diarios/	Observaciones por turno (M/T/N)
Monitoría Hemodinámica	lib/features/monitorias_hemodinamicas/	Signos vitales hora por hora
Balance de Líquidos	lib/features/balance_liquidos/	Administrados y eliminados
Nutrición	lib/features/nutricion/	Registro nutricional con antropometría
Control de Riesgos	lib/features/control_riesgos/	UPP, caídas, anticoagulación, aislamiento
Antibióticos	lib/features/antibioticos/	Tratamientos antibióticos con duración
Catéteres	lib/features/cateteres/	Catéteres venosos y arteriales
Sondas	lib/features/sondas/	Sondas y drenes
Marcapasos	lib/features/marcapasos/	Modo, vía, frecuencia, sensibilidad
Procedimientos Especiales	lib/features/procedimientos_especiales/	Procedimientos con estado y medicación
Control de Sedación (RASS)	lib/features/control_sedacion/	Escala Richmond de -5 a +4
Cambio de Posición	lib/features/control_cambio_posicion/	Registro de cambios de posición

Glasgow	lib/features/monitorias_he modinamicas/glasgow/	Escala de coma de Glasgow
Observaciones Extras	lib/features/observaciones_ extras/	Lab, cultivos, transfusiones, firmas
Reportes PDF	lib/features/reportes/	Generación y exportación de PDF

10. Generación de Reportes PDF

Los reportes PDF se generan usando el paquete pdf (dart/pdf) y se comparten con el paquete printing. El servicio principal es PdfReporteService.

Este reporte contiene toda la información completa del usuario, toda esta información la recopile de la base de datos de Firestore para agregarla en las tablas del documento pdf con opciones de exportar.

10.1 Flujo de Generación

1. ReporteDataCollector recopila todos los datos desde Firestore de forma paralela.
2. PdfReporteService construye el documento PDF usando pw.Document().
3. Cada página se construye con pw.MultiPage() en formato A4 horizontal (landscape).
4. Las fuentes se cargan desde assets usando rootBundle.load().
5. El PDF se guarda como Uint8List y se comparte con Printing.sharePdf().

10.2 Ejemplo de uso

```
final pdfBytes = await PdfReporteService().generateReporte(  
  ingreso: ingreso,  
  registrosDiarios: registros,  
  monitorias: monitorias,  
  // ... demás parámetros  
);  
  
await Printing.sharePdf(  
  bytes: pdfBytes,  
  filename: 'reporte_${nombrePaciente}.pdf',  
);
```

10.3 Fuentes

Las fuentes PlusJakartaSans se cargan desde assets (fonts/Plus_Jakarta/) usando `pw.Font.ttf()` con `rootBundle.load()`. Deben estar declaradas en la sección assets: de `pubspec.yaml` para que `rootBundle.load()` pueda acceder a ellas.

11. Configuración del Entorno de Desarrollo

11.1 Requisitos

- Flutter SDK 3.x instalado (<https://flutter.dev>)
- Dart SDK 3.x (incluido con Flutter)
- Android Studio o VS Code
- Firebase CLI (`npm install -g firebase-tools`)
- Cuenta de Firebase Console con proyecto creado
- Git para control de versiones

11.2 Pasos

Clonar el repositorio

```
git clone [URL del repositorio]
```

```
cd registro_uci
```

Instalar dependencias

```
flutter pub get
```

Configurar Firebase

```
firebase login
```

```
flutterfire configure --project=sabana-digital-prueba
```

Ejecutar en Android

```
flutter run (con dispositivo conectado)
```

Ejecutar en Web

```
flutter run -d chrome
```

Build APK

flutter build apk --release

Build Web + Deploy

flutter build web

firebase deploy --only hosting

12. Buenas Prácticas

12.1 Commits

- Usar commits semánticos: feat:, fix:, refactor:, docs:, chore:
- Commits pequeños y atómicos (un cambio por commit)

12.2 Ramas (Git Flow)

- main — Rama de producción estable
- develop — Integración de características
- feature/* — Nuevas funcionalidades
- fix/* — Correcciones de errores

12.3 Documentación

- Mantener el README.md actualizado
- Documentar providers y métodos públicos
- Seguir la estructura de Clean Architecture

12.4 Pruebas

- Escribir pruebas unitarias para repositorios y controladores
- Ubicación: test/
- Ejecutar: flutter test

13. Posibles Mejoras Futuras

1. **Caché offline:** Sincronización local con Firestore (`firebase_local_cache`) para usar la app sin conexión y que los datos se guarden una vez se restaure la conexión ya que la app funciona 100% con conexión a internet.
2. **Pruebas unitarias:** Cubrir todas las capas con tests automatizados.
3. **Pruebas de integración:** Flutter Driver o Integration Test para flujos completos.
4. **Notificaciones push:** Firebase Cloud Messaging para alertar sobre cambios importantes.
5. **Dashboard/Estadísticas:** Gráficos de evolución de pacientes con `fl_chart`.
6. **Modo oscuro:** Soporte para tema oscuro.
7. **Exportación a Excel:** Además de PDF, exportar datos a formato `.csv/.xlsx`.
8. **Roles personalizados:** Permitir crear roles con permisos configurables.
9. **Internacionalización:** Soporte multilingüe (inglés, portugués).
10. **CI/CD:** Integración continua con GitHub Actions para builds automáticos.

14. Enlaces Útiles

Documentación Flutter: <https://docs.flutter.dev>

Documentación Dart: <https://dart.dev/guides>

Riverpod: <https://riverpod.dev>

Firebase Flutter: <https://firebase.flutter.dev>

Cloud Firestore: <https://firebase.google.com/docs/firestore>

Paquete pdf: <https://pub.dev/packages/pdf>

Paquete printing: <https://pub.dev/packages/printing>

Freezed: <https://pub.dev/packages/freezed>

Firebase Console: <https://console.firebase.google.com>

App Web desplegada: <https://sabana-digital-prueba.web.app>